SafeSurf: A Machine Learning Based Phishing Detection and Protection System with Typosquatting Analysis and a Real-Time Browser Extension

Maheer Arshad
Reg. No. CIIT/SP21-BCT-013/ISB
Department of Computer Science
COMSATS University Islamabad
Islamabad, Pakistan

Nimra Malik
Reg. No. CIIT/SP21-BCT-020/ISB
Department of Computer Science
COMSATS University Islamabad
Islamabad, Pakistan
Email: nimram79@gmail.com

Supervisor: Mr. Muhammad Mustafa Khattak | BS Cyber Security (2021-2025), COMSATS University Islamabad

Phishing remains one of the most effective ways to steal credentials, and the traditional defences, blacklists and fixed rules, cannot keep up with newly created malicious sites [1], [2]. This paper presents SafeSurf, a phishing detection and protection system made of a React web application, a FastAPI back end and a Chrome extension. SafeSurf combines a Random Forest classifier trained on lexical URL features with reputation checks (SSL certificate, Google Safe Browsing, Norton Web Safe, McAfee, Sucuri, local blacklists and IP sets, domain age and a top-one-million site list) and a typosquatting detector built on Levenshtein distance and WHOIS lookups. Users can scan a URL, report suspicious links individually or in bulk, and browse a database of reported URLs, while an extension warns them in real time. The classifier was trained on a balanced dataset of about 450,176 records and reached 87.75% test accuracy. We describe the design and algorithms, support each design choice with published research, and report testing results.

Keywords: phishing detection, machine learning, Random Forest, typosquatting, Levenshtein distance, Chrome extension, web security

Online banking, shopping and public services have made credential theft a profitable business. Malicious URLs host scams and phishing pages that cause losses of billions of dollars every year, so timely detection matters [1]. Blacklists are the classic defence, but they cannot be exhaustive and cannot detect newly generated malicious URLs [1]. Rule-based heuristics detect fresh sites but depend on hand-set thresholds, and no single solution captures every phishing attack because the features that separate phishing from legitimate pages keep changing [2].

SafeSurf addresses this gap with an end-to-end tool. Its objectives are to (i) raise detection accuracy with machine learning, (ii) alert users in real time inside the browser, (iii) raise user awareness through clear verdicts and a reporting workflow, and (iv) keep detection fast and light on computing resources.

II. RELATED WORK AND SCIENTIFIC BACKGROUND

Sahoo et al. survey malicious URL detection and describe it as learning a prediction function from labelled malicious and benign URLs [1]. Ma et al. showed that lexical features of the URL together with host-based features can classify malicious sites without downloading page content [3]. Abu-Nimeh et al. compared several machine learning techniques for phishing detection [4], and Mohammad et al. applied neural networks to website features and released the dataset later hosted by the UCI repository [2], [9]. A University of Central Florida project trained classifiers on that dataset and shipped the chosen model in a Chrome extension [10]; SafeSurf follows the same offline-training, in-browser-prediction pattern but adds a larger balanced dataset, reputation checks, typosquatting analysis, user reporting and an analyst dashboard.

Evidence for the chosen methods

- **Random Forest.** Breiman showed that the generalisation error of a forest of randomised trees converges as trees are added, so more trees do not cause over-fitting [6]. Averaging many trees also suits mixed count and binary URL features.
- **SVM and neural networks.** Support-vector networks use a maximum-margin boundary with a soft margin for overlapping classes [7]; neural networks have been applied directly to phishing prediction [2]. Both were compared with Random Forest.
- **Class balance.** Imbalanced data biases classifiers towards the majority class, and adding data is a standard remedy [12]. We therefore merged two datasets to balance phishing and legitimate URLs.
- **Typosquatting.** Speculators register typo domains in bulk to deploy phishing sites or malware, and an estimated 20% of .com registrations are true typo domains [5]. Edit distance [11] is a standard measure of string similarity.

TABLE I. RELATED SYSTEMS AND SAFESURF'S RESPONSE

System	Weaknesses	SafeSurf response
PhishGuard extension	Static blacklists, basic rules, limited adaptability, high false positives, no real-time protection	Machine learning, real-time analysis, retraining from reports
GuardianShield	Static blacklists, limited adaptability	Machine learning, simple user interface

III. SYSTEM OVERVIEW

SafeSurf has six modules: user dashboard (sign-up, login, profile, reports), machine learning pipeline, phishing detection, typosquatting detection, browser plugin and URL reporting. The front end is built with React.js, the back end with Python and FastAPI, and Firebase provides authentication and storage of user data, reports and activity logs. Scikit-learn [8] is used for model training. Fig. 1 shows the architecture.

Reporting. Users can report one URL or up to 1,000 URLs at once with optional details. Each report is logged, analysed and categorised as legitimate or potentially harmful, shown in the user's dashboard, and kept in a reported-URL database that analysts can review and mark as reviewed; confirmed reports feed later retraining. **Non-functional goals** are reliability (invalid or unreachable URLs are handled without failure), usability (plain-language verdicts and a simple toolbar icon), performance (prediction fast enough to run on every page load) and portability (browser-based clients).

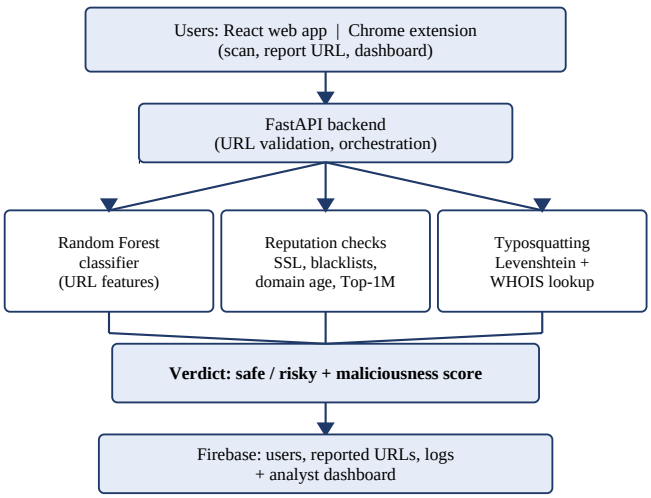

Fig. 1. SafeSurf architecture.

TABLE II. FUNCTIONAL REQUIREMENT GROUPS (38 REQUIREMENTS)

Requirement group	FRs	Scope
Account management	FR-1 to FR-10	Sign-up, password pattern and validation, email and phone verification, forgot password, logout, profile update
Reports and ML pipeline	FR-11 to FR-18	View reports; data collection, preprocessing, feature extraction, model selection, training, evaluation, deployment
Phishing detection	FR-19 to FR-27	SSL checker, Safe Browsing validator, blacklist checker, domain identifier, AI maliciousness predictor, domain age validator, report-database and local blacklist checks
Typosquatting and reporting	FR-28 to FR-34	Typosquatting detection, ML integration, visual similarity analysis, report URL, URL logging, analysis and categorisation
Browser plugin	FR-35 to FR-38	Plugin, toolbar icon, pop-up notifications, settings page

User interface. The web application provides login and sign-up (including Google sign-up), a home page with the URL scan box, separate safe and unsafe result screens, a report-URL page, a typosquatting detector that lists look-alike domains with registrar, country, creation date and name servers, and a database of all reported URLs. The extension popup shows whether phishing was detected on the current page. User data, reports and logs are stored as JSON documents in Firebase.

IV. DATASET AND MACHINE LEARNING

The pipeline has four stages: data collection, dataset processing, classifier training and model optimisation. Reviewers noted that the original dataset was imbalanced, so two datasets of labelled phishing and legitimate URLs were merged (about 450,176 records) and enriched through feature engineering. Lexical features include URL, hostname, path and first-directory lengths, counts of special characters ('-', '@', '?', '%', '.', '=', 'http', 'https', 'www'), digits, letters and directories, plus binary features for an IP address in the URL and a URL shortening service (Algorithm 1).

Algorithm 1: URL feature extraction

Input: list of URLs. **Output:** feature table.
1: **for each** URL u **do**
2: $p \leftarrow$ parse(u); lengths: $|u|$, $|p.\text{host}|$, $|p.\text{path}|$, |first directory|
3: counts of '-', '@', '?', '%', '.', '=', 'http', 'https', 'www'
4: counts of digits, letters and '/' (directories)
5: ip(u) ← -1 if an IPv4/IPv6 pattern is found, else 1
6: short(u) ← -1 if a shortening-service domain is found, else 1
7: **return** table of all features

The classifier is a Random Forest. Each tree h_t is grown on a bootstrap sample with random feature subsets, and the forest predicts the majority vote, $y = \text{mode}\{h_1(x), ..., h_T(x)\}$ [6]. Training follows Algorithm 2; the same code path evaluates SVM (RBF kernel, C = 2) and a one-hidden-layer neural network (100 logistic units) for comparison, and Random Forest was selected, reaching 87.75% test accuracy.

Algorithm 2: Training the phishing classifier

Input: merged labelled dataset. **Output:** saved model and metrics.
1: load dataset; drop rows with missing values
2: split features X and label y (1 = phishing, 0 = legitimate)
3: split 80% train / 20% test (fixed random seed)
4: fit StandardScaler on train; scale train and test
5: $M \leftarrow$ RandomForest(100 trees); fit(M, train)
6: predict test labels; compute accuracy, confusion matrix, precision, recall and F1
7: serialise M (pickle) for the FastAPI service

Performance is measured with accuracy = (TP + TN) / (TP + TN + FP + FN), precision = TP / (TP + FP), recall = TP / (TP + FN) and F1 = 2PR / (P + R), where a false positive is a safe site flagged as phishing. False negatives are costly because a phishing page is trusted, while false positives reduce user confidence, so both are reported through the confusion matrix.

TABLE III. CLASSIFIERS COMPARED

Model	Configuration	Role
Random Forest	100 trees (entropy criterion in the comparison script), scaled features	Deployed model, 87.75% accuracy
SVM	RBF kernel, C = 2.0 [7]	Comparison baseline
Neural network	MLP, 100 hidden units, logistic activation [2]	Comparison baseline

V. DETECTION PIPELINE AND TYPOSQUATTING

When a URL is scanned, the back end gathers independent signals (Algorithm 3). Combining lexical, host-based and reputation evidence follows the survey recommendation to use several feature types [1]. Domain age is checked because newly registered domains are a common phishing signal; SafeSurf expects domains older than three months. Blocklisted IP sets from threat-intelligence services were added so that links hosted on known malicious ranges, including bot-generated ones, are also caught.

Algorithm 3: Phishing detection for a URL

Input: URL u. **Output:** verdict and maliciousness score.
1: validate and normalise u
2: $s_{top} \leftarrow u$ in top-1-million valid sites list
3: $s_{ssl} \leftarrow$ certificate valid?; $s_{tmp} \leftarrow$ temporary domain?
4: $s_{sb} \leftarrow$ Google Safe Browsing, Norton Web Safe lookup
5: $s_{bl} \leftarrow$ McAfee, Sucuri, local blacklist, blocklisted IP sets
6: $s_{age} \leftarrow$ domain age > 3 months?
7: $s_{rep} \leftarrow u$ in reported-URL database?
8: $s_{ml} \leftarrow$ Random Forest probability of phishing
9: aggregate the signals into a verdict and score; log to Firebase

Typosquatting. Similarity between two domains a and b is scaled from 0 to 10: $\text{sim}(a, b) = 10 \times (1 - d(a, b) / \max(|a|, |b|))$, where d is the Levenshtein edit distance [11]. Algorithm 4 generates look-alike permutations of a domain, keeps those with a score above 5, and uses WHOIS to separate registered (verified) from unregistered candidates, returning registrar, country, creation date and name servers. The reverse check compares a suspicious domain with the top-one-million list using a threshold of 7.

Algorithm 4: Typosquatting detection

Input: domain D. **Output:** look-alike domains with status.
1: strip scheme and 'www' from D
2: **if** D not registered (WHOIS) **and** not in top-1M **then return** none
3: $C \leftarrow$ permutations of D (substitution, omission, insertion)
4: $C \leftarrow \{ c \text{ in } C : \text{sim}(D, c) > 5 \}$, at most 5000
5: **for each** c in C **do** WHOIS(c)
6: registered: record registrar, country, creation date, name servers; status = VERIFIED
7: unregistered: status = UNVERIFIED (at most 500)
8: **return** list of results

VI. BROWSER EXTENSION

The Chrome extension (Phishing Detective) runs a content script on every page. It computes 16 lexical and page features, each scored +1 (phishing indicator) or -1 (legitimate): IP address in the URL, long URL, shortened URL, '@' symbol, redirection with '//', hyphen in the domain, multiple subdomains, favicon, images, anchors and scripts loaded from other domains, illegal HTTPS token, invalid form action, mailto links, tampered status bar and iframes. These follow the feature definitions of the UCI Phishing Websites dataset (11,055 URLs, 30 features) [9]. A linear model trained offline supplies one weight per feature, so the browser only evaluates a weighted sum (Algorithm 5), which keeps client computation low in line with objective (iv). The result is sent to the background script, which updates the toolbar icon and shows a pop-up alert.

Algorithm 5: In-browser phishing prediction

Input: current page URL and DOM. **Output:** +1 phishing, -1 legitimate.
1: load weights $w_{1..16}$ learned offline
2: $f_i \leftarrow$ value of feature i in {-1, +1} for i = 1..16
3: $s \leftarrow$ sum of $w_i \times f_i$
4: **if** $s > 0$ **then** label $\leftarrow$ +1 **else** label $\leftarrow$ -1
5: send label to background script; update icon; show pop-up
6: repeat on every page load

VII. IMPLEMENTATION

TABLE IV. TECHNOLOGY STACK

Component	Technology
Front end	React.js 18.2, HTML5, CSS3, Figma mock-ups
Back end	Python, FastAPI, NumPy, urllib3
Machine learning	Scikit-learn (Random Forest, SVM, MLP)
Database and auth	Firebase (authentication, CRUD storage)
Extension	JavaScript content and background scripts, manifest.json

VIII. TESTING AND RESULTS

Testing covered unit, functional and integration levels: login and sign-up (including Google sign-up and password validation), profile editing, logout, URL feature extraction, the machine learning pipeline, phishing detection against several validation sources, typosquatting scans, URL reporting and the browser plugin. All recorded test cases passed. On the held-out test set, Random Forest reached **87.75%** accuracy and was deployed. This is below 90%, so some phishing links will be missed and some safe links flagged, which is why the reputation checks and the user reporting workflow supplement the model.

TABLE V. TEST SUMMARY

Level	Features tested	Result
Unit	Login (valid and invalid), sign-up, Google sign-up, URL feature extraction, edit profile, logout	Pass
Functional	User login and Google sign-up, scanning, reporting, dashboard and report review	Pass
Integration	Phishing detection against validation sources, typosquatting scan, ML pipeline (training, evaluation, retraining, real-time response), browser plugin	Pass

IX. LIMITATIONS AND FUTURE WORK

Current limits: the system supports English only, phishing URLs are scarcer than legitimate ones, and accuracy is not 100%. Future work: expand the dataset with newer phishing and legitimate URLs, add AI-driven threat analysis for emerging trends, support more languages, retrain regularly from user reports, and work with internet service providers for network-level detection.

X. CONCLUSION

SafeSurf combines a Random Forest classifier, reputation checks, typosquatting analysis and a real-time Chrome extension in one system. Published research on blacklist limits [1], feature-based phishing prediction [2], [3] and typosquatting [5] supports the design, and the system gives users immediate feedback while letting them contribute reports that improve detection.

ACKNOWLEDGMENT

The authors thank their supervisor, Mr. Muhammad Mustafa Khattak, for his guidance throughout this project.

REFERENCES

[1] D. Sahoo, C. Liu and S. C. H. Hoi, "Malicious URL detection using machine learning: A survey," arXiv:1701.07179, 2017.

[2] R. M. Mohammad, F. Thabtah and L. McCluskey, "Predicting phishing websites based on self-structuring neural network," *Neural Comput. Appl.*, vol. 25, no. 2, pp. 443-458, 2014.

[3] J. Ma, L. K. Saul, S. Savage and G. M. Voelker, "Beyond blacklists: Learning to detect malicious web sites from suspicious URLs," in *Proc. ACM SIGKDD*, 2009, pp. 1245-1254.

[4] S. Abu-Nimeh, D. Nappa, X. Wang and S. Nair, "A comparison of machine learning techniques for phishing detection," in *Proc. eCrime Researchers Summit*, 2007.

[5] J. Szurdi *et al.*, "The long 'taile' of typosquatting domain names," in *Proc. USENIX Security Symp.*, 2014, pp. 191-206.

[6] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5-32, 2001.

[7] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273-297, 1995.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.

[9] R. M. Mohammad, F. Thabtah and L. McCluskey, "Phishing websites data set," UCI Machine Learning Repository. [Online]. Available: http://archive.ics.uci.edu/ml

[10] A. R. Sura, J. Kini and K. Athrey, "Machine learning approach to phishing detection," Univ. of Central Florida, student project report.

[11] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707-710, 1966.

[12] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263-1284, 2009.